

WET2VO Web Technologies

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

SS 2026

Vorlesung 3

PHP-Grundlagen Teil 2 und Regular Expressions

Nach der grundlegenden Einführung in die Sprache PHP sowie der Demonstration von Stringfunktionen, soll nun das Konzept der Funktionen im Detail beleuchtet werden. Weiters werden Konstanten und Arrays erklärt, bevor ein grundsätzlicher Formularverarbeitungsablauf besprochen und mit Regular Expressions geschlossen wird.

3.1 Funktionen

Eine *Funktion* ist ein benannter Code-Block, der eine bestimmte Aufgabe erfüllt und beliebig oft aufgerufen werden kann. Der PHP-Interpreter muss eine Funktion dabei nur ein Mal abarbeiten, egal wie oft sie schließlich verwendet wird. Funktionen in PHP können, wie in anderen Sprachen auch, *Parameter* (Übergabewerte) haben oder auch Werte zurückliefern. Ob eine Funktion Parameter hat oder etwas zurückgibt, wird bei ihrer Definition festgelegt.

3.1.1 Funktionen definieren (benannte Funktionen)

Eine Funktion wird mit dem Schlüsselwort `function` definiert. Danach folgen der Name der Funktion (Groß- und Kleinschreibung spielt keine Rolle) und zwei Klammern. Diese können optional Parameter enthalten. Der Code, der durch Aufruf der Funktion ausgeführt werden soll, steht in geschwungenen Klammern. Die Syntax lautet also:

```
1 function name($param1, $param2, ...)  
2 {  
3     Anweisung;  
4 }
```

Damit eine Funktion einen Wert zurückgibt, muss innerhalb der Funktion das Schlüsselwort `return`, gefolgt vom zurückzugebenden Wert gesetzt werden. Folgendes Codebeispiel zeigt eine Funktion, die den übergebenen Wert quadriert und zurück gibt:

```
1 function quadrat($x)  
2 {  
3     return $x * $x;  
4 }
```

Auch bei Funktionen kann *Advanced Escaping* zum Einsatz kommen (da eine Funktion durch die geschweiften Klammern auch einen Block erzeugt). Folgendes Konstrukt ist also möglich:

```
1 <?php
2 function noresult()
3 {
4 ?>
5 <p>Die Abfrage lieferte kein Ergebnis!</p>
6 <?php
7 }
8 ?>
```

3.1.2 Funktionsparameter

PHP kennt zwei Arten der Parameterübergabe: Call by Value und Call by Reference.

Call by Value

Parameter, die an Funktionen übergeben werden, sind, wenn sie wie zuvor gezeigt, übergeben werden, reine Eingangsparameter. D. h., es wird eine Kopie des Werts per *Call by Value* übergeben. Das Beispiel demonstriert dies:

```
1 function cbv($x)
2 {
3     return $x += 5;
4 }
5
6 $test1 = 5;
7 $result1 = cbv($test1);
8 echo $result1; // 10
9 echo $test1; // 5
```

Die Funktion `cbv()` wird mit einem Parameter aufgerufen, zählt zu diesem den Wert 5 hinzu und gibt das Ergebnis zurück. Zu Testzwecken wird die Variable `$test1` mit dem Wert 5 angelegt und der Funktion übergeben. Das Ergebnis der Funktion wird in `$result1` gespeichert. Beide Werte werden ausgegeben. `$result1` hat den Wert 10, `$test1` bleibt jedoch unverändert bei 5, da nur eine Kopie an die Funktion übergeben wurde.

Call by Reference

Anstatt der Kopie eines Werts, kann auch eine Referenz an die Funktion übergeben werden. Dies hat zur Folge, dass sich im Falle einer Änderung des Parameters innerhalb der Funktion diese Änderungen auch in der ursprünglichen Variable widerspiegeln. Um einen sogenannten Übergabeparameter zu erzeugen, wird dem Parameter in der Funktionsdefinition ein kaufmännisches Und (`&`) vorangestellt. Das folgende Beispiel illustriert dies:

```
1 function cbr(&$x)
2 {
3     return $x += 5;
4 }
```

```
5
6 $test2 = 5;
7 $result2 = cbr($test2);
8 echo $result2; // 10
9 echo $test2; // 10
```

Die Funktion heißt nun `cbr()` und akzeptiert wieder einen Parameter. Diesem ist jedoch das `&`-Zeichen vorangestellt, somit handelt es sich um einen Übergabeparameter. Der Wert von `$test2` wird mit 5 festgelegt und der Funktion übergeben. Das Ergebnis des Aufrufs wird in `$result2` gespeichert und ist – wie zu erwarten – 10. Bei der Ausgabe von `$test2` hat die Variable nun aber auch den Wert 10, denn der Übergabeparameter wurde in der Funktion verändert und diese Änderung spiegelt sich in der Variable „außerhalb“ der Funktion wider.

3.1.3 Funktionen aufrufen

Der Aufruf von Funktionen wurde in den soeben besprochenen Beispielen bzw. auch bereits in Abschnitt 2.3.9 demonstriert. Der Name der Funktion, gefolgt von Klammern und ggf. Argumenten genügt. Liefert die Funktion einen Wert zurück, kann dieser einer Variable zugewiesen werden.

Häufige Fehler sind das Weglassen des `$`-Zeichens für Argumente oder auch das Setzen des `&`-Zeichens (weil Call by Reference) gewünscht ist. Dies muss jedoch bei der Definition der Funktion geschehen, nicht bei deren Aufruf. Im folgenden einige Beispiele:

```
1 quadrat(5); // Ruft quadrat() mit dem Wert 5 auf
2 cbv($value); // Ruft cbv() mit der Variable $value auf
3 $result = cbr(5); // Ruft cbv() mit Wert 5 auf und weist Ergebnis an $result zu
4 irgendwas(); // Ruft irgendwas() ohne Parameter auf
```

Häufige Fehler beim Aufruf sind das Vergessen des `$`-Zeichens bei Variablen als Argumenten oder der Versuch, das `&`-Zeichen (für Call by Reference) beim Aufruf mitzugeben. Letzteres darf ausschließlich in der Definition der Funktion stehen.

First-Class Callable Syntax

Seit PHP 8.1 gibt es die *First-Class Callable Syntax*. Mit der Notation `name(...)` kann eine Funktion referenziert werden, ohne sie sofort auszuführen. Dies ist besonders wichtig, wenn Funktionen als Argumente an andere Funktionen übergeben werden oder – wie in Abschnitt 3.1.6 gezeigt – im *Pipe-Operator* zum Einsatz kommen.

```
1 // Referenz auf die Funktion speichern (nicht ausführen!)
2 $funcRef = quadrat(...);
3
4 // Späterer Aufruf über die Referenz
5 echo $funcRef(10); // 100
```

3.1.4 Default-Parameter

PHP erlaubt es, Parameter von Funktionen mit *vorgegebenen Werten* zu belegen. Dies ist dann interessant, wenn Parameter optional sein sollen. Soll z. B. eine Sortierfunktion geschrieben werden, die Wörter von A–Z sortiert, wird als Parameter ein Array mit diesen Wörtern übergeben. Ein optionaler zweiter, boolescher Parameter soll ermöglichen,

dass die Sortierreihenfolge auf Z–A geändert wird. Wird der Parameter mit **false** explizit angegeben, wird die Reihenfolge umgedreht. Wird er weggelassen, wird er einfach mit **true** angenommen.

Dies ist möglich, indem Parametern in der Funktionsdefinition einfach ein Wert zugewiesen wird. Dabei ist zu beachten, dass ab dem ersten vorbelegten Parameter auch alle weiteren vorbelegt werden müssen. Das folgende, einfache Beispiel demonstriert dies:

```
1 function defaultParams($a, $b = "b", $c = "c")
2 {
3     echo "<p>$a$b$c</p>";
4 }
5
6 defaultParams("a"); // abc
7 defaultParams("a", "x"); // axc
8 defaultParams("a", "x", "o"); // axo
```

Der erste Parameter (**\$a**) ist nicht vorbelegt, somit muss er beim Funktionsaufruf immer angegeben werden. Der zweite (**\$b**) und dritte Parameter (**\$c**) sind jeweils vorbelegt und können weggelassen werden. Ist dies der Fall, werden die vorbelegten Werte angenommen. Wird für die Parameter ein Wert mit übergeben, so wird dieser verwendet. Die Werte werden von links nach rechts zugeordnet. Wird also die Funktion wie in Zeile 7 mit zwei Parametern aufgerufen, so wird der zweite Parameter als **\$b** angenommen.

3.1.5 Anonyme Funktionen und Arrow Functions

Normalerweise haben Funktionen einen Namen, um sie später wiederholt aufrufen zu können. Es gibt jedoch Anwendungsfälle, in denen eine Funktion nur an einer einzigen Stelle als kurzes Hilfswerkzeug benötigt wird. In diesem Fall spricht man von *anonymen Funktionen* (auch Closures genannt).

Seit PHP 7.4 existieren zudem *Arrow Functions*. Sie sind eine verkürzte Schreibweise für anonyme Funktionen, die aus nur einem Ausdruck bestehen. Sie beginnen mit dem Schlüsselwort **fn** und geben das Ergebnis nach dem Pfeil (**=>**) automatisch zurück.

Über diese Schreibweise ist es möglich, einer Variable eine Funktion zuzweisen: die Variable „enthält“ also eine Funktion. Über die Angabe des Variablennamens wird somit die Funktion aufgerufen.

```
1 // 1. Klassische anonyme Funktion
2 $callback = function ($name){
3     return "Hallo $name";
4 };
5
6 echo "<p>{$callback("Igor")}</p>"; // Hallo Igor
7
8 // 2. Moderne Arrow Function (kürzer, gleiches Ergebnis)
9 $arrow = fn($name) => "Hallo $name";
10
11 echo "<p>{$arrow("Igor")}</p>"; // Hallo Igor
```

Hier zeigt sich auch ein weiterer Vorteil dieser Syntax: Der Funktionsaufruf kann, da es ja „nur“ der Variablenname ist, in einem String mit doppelten Anführungszeichen in der Complex Curly Syntax stehen. Beim Aufruf einer benannten Funktion muss der String unterbrochen und mit **.** verbunden werden.

3.1.6 Der Pipe-Operator

Mit PHP 8.5 wurde der *Pipe-Operator* (`|>`) eingeführt. Er dient dazu, das Ergebnis einer Operation linear an die nächste Funktion weiterzureichen. Dies verhindert tief verschachtelte Funktionsaufrufe, die „von innen nach außen“ gelesen werden müssten, und macht den Datenfluss von oben nach unten (bzw. links nach rechts) sichtbar.

Der Operator erwartet auf der rechten Seite ein *Callable* (etwas Aufrufbares). Benötigt die Zielfunktion mehr als nur das eine übergebene Argument, muss eine Arrow Function genutzt werden. In dieser wird dann spezifiziert, wo der übergebene Wert verwendet wird.

Im folgenden Beispiel werden bei einem Input-String zunächst mit `mb_trim()` die Leerzeichen am Anfang und Ende entfernt, das Ergebnis in Großbuchstaben umgewandelt und schließlich nur die ersten fünf Buchstaben entnommen:

```

1 $input = "  hällö welt  ";
2
3 // Verschachtelter Funktionsaufruf (ggf. schwer lesbar)
4 $res1 = mb_substr(mb_strtoupper(mb_trim($input)), 0, 5);
5 echo "<p>$res1</p>";
6
7 // Pipe-Operator (linearer Datenfluss)
8 $res2 = $input
9         |> mb_trim(...)
10        |> mb_strtoupper(...)
11        |> (fn($x) => mb_substr($x, 0, 5));
12
13 echo "<p>$res2</p>"; // HÄLLO

```

3.1.7 Typ- und Rückgabewertsdeklarationen

Das dynamische Typkonzept von PHP bietet sehr viel Flexibilität im Zusammenhang mit Funktionen. So können Funktionsparameter jeglichen Datentyp aufnehmen, ebenso kann mit der `return`-Anweisung jeder beliebige Typ zurückgegeben werden. Es gibt jedoch auch die Möglichkeit, diese Freiheiten zugunsten von mehr Kontrolle einzuschränken.

Typdeklarationen

Durch eine *Typdeklaration* wird bei der Erstellung einer Funktion festgelegt, dass diese nur Parameter des angegebenen Typs zur Laufzeit akzeptiert. Um dies zu erreichen, wird der Typ einfach vor den Parameter gestellt. Gültige Angaben sind alle von PHP unterstützten Typen, etwa die skalaren Typen `int`, `float`, `bool` und `string`, `object` sowie Klassennamen (siehe Vorlesung 4), `array` oder auch die Schlüsselwörter `callable`¹ (die in Abschnitt 3.1.5 erwähnten anonymen Arrow-Functions, die auch beim Pipe-Operator in Abschnitt 3.1.6 benötigt werden) und `iterable`² (für Arrays und Klassen, die das Interface `Traversable` implementieren).

Im folgenden Beispiel wird eine Funktion definiert, die einen Parameter vom Typ `int` erwartet:

¹<https://www.php.net/manual/de/language.types.callable.php>

²<https://www.php.net/manual/de/language.types.iterable.php>

```
1 function isInt(int $number)
2 {
3     echo "<p>Eine Zahl: $number</p>";
4 }
```

Diese Funktion soll nun mit drei unterschiedlichen Parametern aufgerufen werden:

```
1 isInt(3);
2 isInt("3");
3 isInt("hallo");
```

Der Aufruf in Zeile 1 funktioniert immer, da eine Ganzzahl übergeben wird. Der Aufruf in Zeile 2 funktioniert ebenso, da durch das lose Typkonzept (erfolgreich) versucht wird, den String in einen Integer zu konvertieren. Der Aufruf in Zeile 3 hingegen schlägt immer fehl und produziert einen `TypeError`-Fehler.

Um ein noch strengeres Vorgehen zu erzwingen und auch den Aufruf von Zeile 2 fehlschlagen zu lassen, kann als erste Anweisung in der PHP-Datei (direkt nach `<?php`) die Deklaration `declare(strict_types=1);` gesetzt werden. Sie verhindert die automatische Umwandlung.

Nullable Types: Seit PHP 7.1 besteht die Möglichkeit, die Typdeklaration „nullable“ zu machen, d. h. dass neben dem angegebenen Datentyp auch `null` übergeben werden kann. Dazu wird vor den Typ ein Fragezeichen (?) gestellt:

```
1 function isIntOrNull(?int $number)
2 {
3     echo "<p>Eine Zahl: $number</p>";
4 }
```

Bis PHP 8.4 war es jedoch (unverständlichlicherweise) möglich, für jeden Parameter `null` als Default-Wert anzugeben, auch wenn kein Fragezeichen vorgestellt war. Mit dieser Version wurde dieses Schlupfloch jedoch geschlossen (deprecated).

Rückgabewertsdeklarationen

Nach demselben Prinzip funktionieren *Rückgabewertsdeklarationen*, die mit PHP 7 eingeführt wurden. Sie erlauben anzugeben, welchen Typ eine Funktion zurückgibt. Der Typ wird dabei durch einen Doppelpunkt getrennt am Ende des Funktionskopfes angegeben. Die folgende Funktion zum Addieren von zwei Werten gibt ihr Ergebnis als Ganzzahl zurück:

```
1 function getSum($a, $b): int
2 {
3     return $a + $b;
4 }
```

Auch hier beeinflusst `declare(strict_types=1);` die Rückgabe. Folgende drei Aufrufe illustrieren dies:

```
1 getSum(3, 4);
2 getSum(3.0, 4.0);
3 getSum(3.0, 2.5);
```

Das Ergebnis des Aufrufs in Zeile 1 ist immer 7, da zwei Ganzzahlen addiert werden und funktioniert immer. Die Zeilen 2 und 3 funktionieren nur, wenn die Angabe von

`declare(strict_types=1);` nicht gesetzt ist, da hier Gleitkommazahlen addiert werden und daher auch das Ergebnis eine Gleitkommazahl ist (allerdings in eine Ganzzahl konvertiert wird). Vorsicht ist daher in Zeile 3 geboten: Hier liefert die Funktion – auch wenn sie funktioniert – ein falsches Ergebnis. Durch die automatische Umwandlung von `float` auf `int` werden die Kommastellen abgeschnitten, weshalb als Ergebnis 5 (anstatt 5,5) zurückgegeben wird. PHP merkt dies mit einer Deprecation Notice an, um Entwickler*innen zu warnen, dass dies unvorhersehbare Konsequenzen haben kann.

Nullable Return-Types: Auch Rückgabedeklarationen können seit PHP 7.1 „nullable“ sein, d.h. es kann der angegebene Typ oder `null` zurückgegeben werden. Wiederum wird das Fragezeichen (?) als Ausdruck vor den Rückgabewert gestellt:

```
1 function getSumOrNull($a, $b): ?int
2 {
3     return ($a + $b) > 0 ? $a + $b : null;
4 }
```

Void Funktionen: Ebenfalls kann für Funktionen explizit der Rückgabotyp `void` angegeben werden. Bei diesen muss das `return`-Statement entweder komplett entfallen oder darf keinen Wert zurückgeben (d.h. muss alleine stehen):

```
1 function getNothing(): void
2 {
3     echo "<p>Hier wird nichts zurückgegeben.</p>";
4     return; // Optional, hier besser gleich weglassen
5 }
```

Never Return-Type: Seit PHP 8.1 kann der Rückgabotyp `never` angegeben werden. Dieser ist dann sinnvoll, wenn eine Funktion nie zurückkehrt. Dies ist dann der Fall, wenn in ihr die Ausführung der PHP-Datei etwas mit `die()` oder `exit()` beendet wird, oder eine Ausnahme (Exception) wirft. Ersteres ist etwa der Fall, wenn eine Weiterleitung auf eine andere Seite durchgeführt wird (mehr dazu in Vorlesung 8):

```
1 function redirect(string $uri): never
2 {
3     header("Location: $uri");
4     exit();
5 }
```

Rückgabewerte `true`, `false` und `null`: Seit PHP 8.2 können `true`, `false` und `null` ebenfalls als Rückgabewerte von Funktionen angegeben werden. Zuvor war dies nur gemeinsam in Union Types (s. unten) oder als Nullable Type möglich. Nun ist eine derartige Deklaration ebenfalls möglich (wenngleich wohl nicht sehr sinnvoll):

```
1 function alwaysFalse(): false
2 {
3     return false;
4 }
```

Union Types

Mit PHP 8.0 wurde es möglich, für Parameter und Rückgabewerte einen sogenannten *Union Type*, also eine Auswahl an zwei oder mehreren Typen, anzugeben. Dies geschieht mittels `|`-Zeichen. Es können dabei beliebig viele Typen aneinandergereiht werden:

```
1 function getSum(int|float $a, int|float $b): int|float
2 {
3     return $a + $b;
4 }
```

Beide Parameter dürfen von Typ `int` oder `float` sein. Sind beide Ganzzahlen, wird auch eine Ganzzahl zurückgegeben, sobald mindestens eine Gleitkommazahl dabei ist, wird auch eine solche zurückgegeben. Andere Datentypen sind nicht möglich.

3.1.8 Gültigkeitsbereich von Variablen

Auch PHP kennt das Konzept von globalen und lokalen Variablen. Mit diesem müssen sich Entwickler*innen meist im Zusammenhang mit Funktionen auseinandersetzen. *Globale Variablen* gelten in der gesamten PHP-Datei, *lokale Variablen* gelten innerhalb des Blocks, in dem sie definiert wurden. PHP verfolgt im Detail jedoch ein etwas anderes Konzept als andere Sprachen (z. B. JavaScript):

Variablen, die *in* einer Funktion definiert wurden, sind *außerhalb* dieser Funktion nicht zugreifbar. Variablen, die *außerhalb* einer Funktion definiert wurden, sind standardmäßig *in* der Funktion ebenfalls nicht zugreifbar.

Für lokale Variablen (die in einer Funktion definiert wurden), ist das Konzept das übliche. Sie gelten nur in der Funktion und nirgendwo sonst. Für globale Variablen jedoch gilt umgekehrt genau das Gleiche. Sie werden außerhalb aller Funktionen definiert, gelten aber dann normalerweise innerhalb der Funktionen *nicht*. Hier unterscheidet sich PHP stark von anderen Sprachen wie Java oder JavaScript, wo globale Variablen auch in Funktionen zugreifbar sind.

Um in PHP nun eine globale Variable auch innerhalb einer Funktion verwenden zu können, muss sie in der Funktion zunächst mit dem Schlüsselwort `global` explizit als globale Variable deklariert werden. Ab diesem Zeitpunkt ist dann die Verwendung wieder dieselbe, wie in anderen Sprachen.

Das folgende Beispiel zeigt die Verwendung einer lokalen Variable:

```
1 function fooLokal(): void
2 {
3     $x = "Lokal";
4 }
5
6 $x = "Global";
7 fooLokal();
8 echo $x; // Global
```

Die Funktion `fooLokal()` legt eine lokale Variable `$x` an und weist ihr den Wert „Lokal“ zu. Vor dem Aufruf wird außerhalb der Funktion eine Variable `$x` angelegt, die den Wert „Global“ erhält. Dann wird `fooLokal()` aufgerufen und `$x` ausgegeben. Die Variable hat nach wie vor den Wert „Global“.

Im Gegensatz dazu nun ein Beispiel, bei dem in der Funktion die globale Variable verändert wird:

```
1 function fooGlobal(): void
2 {
3     global $x;
4     $x = "Jetzt global";
5 }
6
7 $x = "Global";
8 fooGlobal();
9 echo $x; // Jetzt global
```

Die Funktion heißt nun `fooGlobal()`. Zunächst wird innerhalb die Variable `$x` mit dem Schlüsselwort `global` als globale Variable deklariert. Somit ist festgelegt, dass diese Variable auch außerhalb Änderungen hervorrufen kann. Die Variable erhält dann den Wert „Jetzt global“. Zu Testzwecken wird wieder eine globale Variable `$x` mit dem Wert „Global“ angelegt. Danach wird `fooGlobal()` aufgerufen und der Wert von `$x` ausgegeben. Dieser ist nun „Jetzt global“, da die Funktion die globale Variable `$x` verändert hat.

Da in PHP Variablen immer implizit deklariert werden, kann das `global` Schlüsselwort auch mit einer Variable verwendet werden, die außerhalb der Funktion noch gar nicht existiert. Sie wird dann in der Funktion einfach direkt als globale Variable angelegt.

Statische Variablen

PHP kennt (wie C oder Java) auch das Konzept der *statischen Variablen* in Funktionen. Diese werden nur einmal initialisiert und ihr Wert bleibt zwischen allen Funktionsaufrufen erhalten. Um eine Variable als statisch zu definieren, wird ihr das Schlüsselwort `static` vorangestellt. Das folgende Beispiel illustriert dies:

```
1 function counter(): int
2 {
3     static $count = 0;
4     return $count++;
5 }
6
7 for ($i = 0; $i < 5; $i++) {
8     echo counter();
9 }
```

Die Variable `$count` wird hier statisch angelegt und beim ersten Aufruf der Funktion `counter()` mit dem Wert 0 belegt. Dieser Wert wird zurückgegeben und `$count` wird erhöht. Eine normale (lokale) Variable würde hier zerstört werden, die statische „lebt“ aber weiter. Beim nächsten Aufruf der Funktion hat `$count` den Wert 1, welcher wiederum zurückgegeben und dann erhöht wird. Somit gibt die Schleife 01234 aus.

3.2 Konstanten

Eine *Konstante* ist ein Bezeichner für einen einfachen Wert (Boolean, Integer, Double oder String), der einmal definiert wird und danach nicht mehr geändert werden kann.

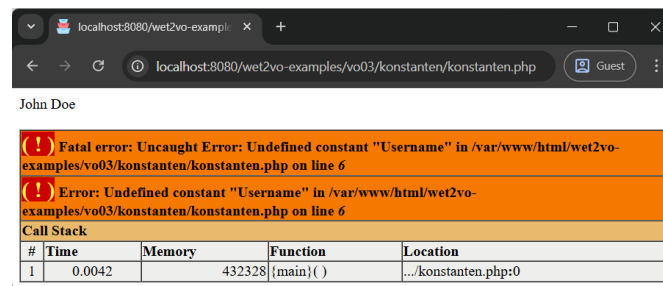


Abbildung 3.1: In der ersten Zeile wird die Konstante `USERNAME` ausgegeben. Danach folgt der Fehler, dass keine Konstante mit dem Namen (bzw. der Schreibweise) `Username` existiert und bricht ab.

Konstanten können selbst definiert werden, PHP stellt jedoch auch eine Reihe von konstanten Werten vorab zur Verfügung. Konstanten sind überall gültig, d. h. sie können auch in Funktionen verwendet werden, ohne vorher speziell deklariert werden zu müssen.

3.2.1 Benannte Konstanten

Konstanten können in PHP auf zwei Arten angelegt werden. Mit der Funktion `define()` oder dem Schlüsselwort `const`.

Konstanten mit `define()`

Die Funktion `define($name, $value)`³ hat zwei Parameter: den Namen oder Identifikator (`$name`) und den Wert (`$value`), der durch die Konstante repräsentiert werden soll. Der Name einer Konstante kann wie der einer Variablen gewählt werden, das `$`-Zeichen entfällt jedoch. Zudem werden Konstantennamen per Konvention in Großbuchstaben gesetzt. Ein Beispiel:

```
1 define("USERNAME", "John Doe");
2 echo USERNAME; // John Doe
```

Zunächst wird die Konstante mit dem Namen `USERNAME` definiert. Sie bekommt den Wert „John Doe“ zugewiesen. Mit `echo` wird dann ihr Wert ausgegeben.

Die Groß- und Kleinschreibung bei Konstanten ist unbedingt einzuhalten. Während vor PHP 8.0 noch eine Warnung bei falscher Schreibweise (z. B. bei `echo Username` anstatt `echo USERNAME`) angezeigt wurde, führt diese nun zu einem Fehler und beendet das Script (siehe Abbildung 3.1). Falsch geschriebene (d. h. in dieser Form so nicht existierende) Variablen erzeugen übrigens nach wie vor nur eine Warnung.

Neben der Groß- und Kleinschreibung sind die Anführungszeichen ein häufiges Problem bei Konstanten. Beim Definieren sind sie anzugeben, bei der Verwendung (z. B. mit `echo`) nicht.

³<https://www.php.net/manual/de/function.define.php>

Konstanten mit `const`

Konstanten können ebenfalls mit dem Schlüsselwort `const` definiert werden. Auch hierbei entfällt das `$`-Zeichen und eine Schreibweise in Großbuchstaben empfiehlt sich.

```
1 const USER = "Jane Doe";  
2 echo USER; // Jane Doe
```

Das Schlüsselwort `const` war ursprünglich für Klassenkonstanten (siehe Vorlesung 4) vorgesehen, kann aber seit PHP 5.3 auch außerhalb von Klassen verwendet werden. Seine Syntax ist klarer und sollte in der Regel bevorzugt werden. Wichtig ist jedoch, dass Konstanten mit `const` nur auf der obersten Ebene in einem Skript verwendet werden können. Eine Definition z. B. innerhalb einer `if`-Bedingung ist nicht möglich. Hierfür muss auf `define()` zurückgegriffen werden:

```
1 if (!defined("FOO")) {  
2     define("FOO", "bar");  
3     // const FOO = "bar"; -> funktioniert hier nicht  
4 }
```

Konstanten überprüfen

Um zu testen, ob eine benannte Konstante existiert, kann (wie gerade zuvor gesehen) `defined($name)`⁴ verwendet werden. Zu beachten ist, dass diese Funktion nur mit Konstanten funktioniert, nicht mit normalen Variablen. Es ist jedoch unerheblich, ob die Konstante mittels `define()` oder `const` angelegt wurde. Der Name der zu überprüfenden Konstanten muss unter Anführungszeichen gesetzt werden. Das Ergebnis der Überprüfung ist `true` oder `false`.

```
1 if (defined("USERNAME")) {  
2     echo "<p>Welcome " . USERNAME . "!\</p>";  
3 }
```

Wie im Beispiel ersichtlich, werden Konstanteninhalte selbst in Strings mit doppelten Anführungszeichen nicht ersetzt. Sie müssen an den String angehängt werden.

3.2.2 Vordefinierte Konstanten

PHP stellt eine Reihe von Konstanten bereit, die immer zur Verfügung stehen und einige praktische Informationen behalten können. Man kann sie grob in drei Kategorien unterteilen:

- magische Konstanten,
- Core-Konstanten und
- Standard-Konstanten.

Magische Konstanten

Magische Konstanten erhalten ihren Namen von der Tatsache, dass sie unterschiedliche Werte haben, abhängig davon, wo sie eingesetzt bzw. aufgerufen werden. Man erkennt

⁴<https://www.php.net/manual/de/function.defined.php>

sie an der Schreibweise: Sie beginnen und enden (bis auf eine Konstante) mit einem doppelten Unterstrich (`__`). Es gibt neun magische Konstanten:

- `__LINE__`: Die aktuelle Zeilennummer in einer Datei.
- `__FILE__`: Der vollständige Pfad- und Dateiname einer Datei.
- `__DIR__`: Der Name des Verzeichnisses, in dem sich die aktuelle Datei befindet.
- `__FUNCTION__`: Der Name der aktuell aufgerufenen Funktion.
- `__CLASS__`: Der Name der aktuellen Klasse (siehe Vorlesung 4).
- `__TRAIT__`: Der Name des aktuellen Traits (siehe Vorlesung 4).
- `__METHOD__`: Der Name der aktuellen Klassenmethode (siehe Vorlesung 4).
- `__PROPERTY__`: Der Name einer Eigenschaft in einem Property-Hook (siehe Vorlesung 4).
- `__NAMESPACE__`: Der Name des aktuellen Namespace (siehe Vorlesung 4).
- `ClassName::class`: Der voll qualifizierte Name einer Klasse (siehe Vorlesung 4).

Core-Konstanten

Diese Konstanten werden vom PHP-Core (also von der Engine) definiert. Sie enthalten Informationen über Versionen, verwendete Betriebssysteme, maximale Größe eines Integers am aktuellen System etc. Im Folgenden werden einige aufgelistet, eine komplette Liste findet sich unter <https://www.php.net/manual/de/reserved.constants.php>.

- `PHP_VERSION`: Die aktuelle PHP-Version.
- `PHP_OS`: Das Betriebssystem, auf dem PHP gerade läuft.
- `PHP_INT_MAX`: Der größtmögliche Integerwert im System.
- `PHP_INT_SIZE`: Die Bytegröße eines Integers im System.
- ...

Standard-Konstanten

Standard-Konstanten sind eine Reihe weiterer vordefinierter Konstanten, die von den mit PHP mitgelieferten Erweiterungen stammen. Zu ihnen zählen etwa mathematische Konstanten wie Pi oder die Eulersche Zahl. Aufgelistet sind sie auf der Seite der jeweiligen Extension⁵.

- `M_PI`: Die Zahl Pi (π).
- `M_E`: Die Eulersche Zahl (e).
- ...

3.3 Arrays

Arrays sind Kollektionen von verschiedenen Werten, die geordnet abgebildet werden. Diese Abbildung erfolgt über Schlüssel/Wert Paare. Der Schlüssel kann entweder ein Integer – in diesem Fall spricht man von einem indizierten Array – oder ein String sein.

⁵Eine Liste aller Erweiterungen findet sich unter <https://www.php.net/manual/de/extensions.membership.php#extensions.membership.core>.

Arrays mit Strings als Schlüssel werden assoziative Arrays genannt. Die Werte können in beiden Fällen beliebig sein.

Arrays sind auf vielfältige Arten und Weisen verwendbar und daher sehr gebräuchlich. Es gibt viele PHP-Funktionen, die mit Arrays arbeiten (z. B. sortieren), aber auch Funktionen, die Arrays als Parameter akzeptieren.

3.3.1 Erzeugen von Arrays

Arrays werden mit der Kurzsyntax `[item1, item2, ...]` erzeugt. Alternativ ist auch das Sprachkonstrukt `array()` möglich, es gilt jedoch mittlerweile als veraltet. Einige Beispiele dazu:

```
1 $a1 = []; // Leeres Array
2 $a2 = ["a", "b", "c"]; // Indiziertes Array
3 $a3 = [3, "Hallo", 5.1]; // Gemische Datentypen als Werte
```

Werden Arrays auf diese Art und Weise angelegt, entstehen automatisch *indizierte* Arrays mit (Integer) Schlüsseln von 0 bis $n - 1$. Sollen diese explizit vergeben werden, kann zur Syntax für assoziative Arrays gegriffen werden. Dabei wird zwischen Schlüssel und Wert ein Pfeil (`=>`) gesetzt. Die so definierten Schlüssel können auch Lücken aufweisen.

```
1 $a4 = [0 => "a", 1 => "b", 2 => "c"]; // Explizit zugewiesene Schlüssel
2 $a5 = [0 => "a", 3 => "b", 4 => "c"]; // Zuweisung mit Lücken
```

Auf diese Art und Weise werden auch *assoziative* Arrays mit Strings als Schlüssel erzeugt:

```
1 $a6 = ["Vorname" => "John", "Nachname" => "Doe", "Alter" => "34"]; // Assoz. Array
2 $a7 = ["farbe" => "rot", "form" => "rund", 3]; // Gemischtes Array
```

Wie in der zweiten Zeile ersichtlich, kann auch ein gemischtes Array erzeugt werden. Der Zugriff auf die Zahl 3 erfolgt hierbei über den Index 0.

Arrays werden aufgrund der Lesbarkeit auch gerne in einer mehzeiligen Schreibweise deklariert. In diesem Fall steht die öffnende Klammer in der Zeile mit dem `=`-Zeichen und die schließende Klammer steht in der nächsten Zeile nach dem letzten Wert. Jede Zeile schließt mit einem Komma, auch die letzte (Trailing Comma):

```
1 $a8 = [
2     "multi",
3     "line",
4     "declaration",
5 ];
```

3.3.2 Zugriff auf Array-Elemente

Um auf einzelne Elemente eines Arrays zuzugreifen, wird der Schlüssel in eckigen Klammern angegeben. Dieser kann ein Integer bei indizierten Arrays oder ein String bei assoziativen Arrays sein. Der erste Indexwert ist immer 0.

```
1 $val1 = $a2[1]; // b
2 $val2 = $a4[0]; // a
3 $val3 = $a6["Vorname"]; // John
4 $val4 = $a7["farbe"]; // rot
5 $val5 = $a7[0]; // 3
```

Vorsicht: Würde mit `$a6[Vorname]` (also ohne Anführungszeichen) zugegriffen, so würde der Zugriff über den Wert der Konstante `Vorname` erfolgen, sofern eine solche definiert wurde.

3.3.3 Erweitern von Arrays

Arrays können in PHP jederzeit erweitert werden. Um einem indizierten Array einen neuen Wert anzuhängen, werden nur die Array-Klammern, aber keine explizite Position angegeben:

```
1 $arr1 = [7, 4, 3]; // Stellen 0, 1, 2 sind belegt
2 $arr1[] = 14; // Landet an Stelle 3
```

Es kann jedoch auch explizit ein Schlüssel angegeben werden, an dem der neue Wert eingefügt werden soll:

```
1 $arr1 = [7, 4, 3]; // Stellen 0, 1, 2 sind belegt
2 $arr1[7] = 14; // Landet an Stelle 7
3 $arr1["test"] = 5; // Landet an Stelle "test"
```

Eine Ausgabe mit `var_dump($arr1);` zeigt dies:

```
/var/www/html/wet2vo-examples/vo03/arrays/arrays.php:25:
array (size=5)
  0 => int 7
  1 => int 4
  2 => int 3
  7 => int 14
  'test' => int 5
```

3.3.4 Arrays durchlaufen

Arrays können ganz normal mit `while`- oder `for`-Schleifen durchlaufen werden. Einfacher ist jedoch die `foreach`-Schleife. Sie durchläuft immer das gesamte Array und gibt in jedem Durchlauf den aktuellen Wert in einer Variable zurück. Die prinzipielle Syntax sieht wie folgt aus:

```
1 foreach (Array as Wert) {
2     Anweisung;
3 }
```

Der erste Parameter ist also das Array, das durchlaufen werden soll. Der zweite Parameter erhält bei jedem Durchlauf den aktuellen Wert. Sein Name kann beliebig gewählt werden. Hier ein konkretes Beispiel:

```
1 $array1 = [3, 14, 4, 9];
2 foreach($array1 as $value) {
3     echo $value; // Ausgabe: 31449
4 }
```

Liegt ein assoziatives Array vor, existiert eine zweite Syntax, die es ermöglicht, auch den Schlüssel in einer Variable auszugeben:

```
1 foreach (Array as Schlüssel => Wert) {
2     Anweisung;
3 }
```

Der erste Parameter ist wiederum das zu durchlaufende Array, dann folgen zwei Variablen für den Schlüssel und den dazugehörigen Wert, deren Namen wieder frei gewählt werden können. In jedem Durchlauf sind darin die aktuellen Werte enthalten und können verarbeitet werden. Ein Beispiel:

```
1 $array2 = ["eins" => 1, "zwei" => 2, "drei" => 3];
2 foreach($array2 as $key => $value) {
3     echo "$key: $value"; // Ausgabe: eins: 1zwei: 2drei: 3
4 }
```

3.4 Funktionen der PHP-Bibliothek

Im Folgenden sollen, wie bereits in Vorlesung 2 für Strings geschehen, einige Funktionen aus der *PHP-Bibliothek* aufgezeigt werden, die für das Arbeiten mit Variablen und Arrays interessant und praktisch sind.

3.4.1 Funktionen für Variablen

Für Variablen gibt es einige praktische Funktionen, um deren Vorhandensein oder Typ zu überprüfen.

Auf Vorhandensein einer Variable prüfen

Die Funktion `isset($var [, $var2, ...])`⁶ überprüft, ob eine oder mehrere Variablen vorhanden sind und gibt `true` oder `false` zurück. Die Funktion akzeptiert beliebig viele Parameter. Sind mehrere angegeben, gibt sie nur dann `true` zurück, wenn alle angegebenen Variablen auch vorhanden sind.

```
1 $a = "hallo";
2 $b = "welt";
3 isset($a); // true
4 isset($a, $b); // true
5 isset($c); // false
6 isset($a, $c); // false
```

Seit PHP 7 kann statt einer Überprüfung mittels `isset` vor der Verwendung einer Variable auch der Null Coalescing Operator (siehe Abschnitt 2.3.6) verwendet werden.

Variablen löschen

Die Funktion `unset($var [, $var2, ...])`⁷ löscht eine (oder mehrere) Variablen. Die Variable existiert danach nicht mehr, eine Überprüfung mit `isset()` ergibt also `false`.

```
1 $a = "hallo";
2 if (isset($a)) { ... } // true
3 unset($a);
4 if (isset($a)) { ... } // false
```

⁶<https://www.php.net/manual/de/function.isset.php>

⁷<https://www.php.net/manual/de/function.unset.php>

Typ-Prüfungen für Variablen

Den Typ einer Variable zu kennen, kann oft wichtig sein. PHP bietet dafür verschiedene Funktionen an. Sie alle geben `true` oder `false` zurück:

- `is_array($var)`:⁸ Überprüft, ob eine Variable ein Array ist.
- `is_bool($var)`:⁹ Überprüft, ob es sich um eine boolesche Variable handelt.
- `is_float($var)`:¹⁰ Überprüft, ob es sich um eine Gleitkommazahl handelt.
- `is_int($var)`:¹¹ Überprüft, ob es sich um eine Ganzzahl handelt.
- `is_null($var)`:¹² Überprüft, ob eine Variable `null` enthält.
- `is_numeric($var)`:¹³ Überprüft, ob eine Variable eine Zahl oder ein numerischer String (ein String, der nur aus Zahlen besteht) ist.
- `is_string($var)`:¹⁴ Überprüft, ob es sich um einen String handelt.

Dazu einige Beispiele:

```
1 $a = "hallo";
2 $b = 34;
3 $c = "1234";
4 is_array($a); // false
5 is_bool($a); // false
6 is_float($b); // false
7 is_int($b); // true
8 is_null($c); // false
9 is_numeric($c); // true
10 is_string($c); // true
```

Ausgabe von Variablen-Inhalten

Inhalte von einfachen Variablen können mit `echo` oder `print()` einfach im Browser ausgegeben werden. Gerade aber bei Arrays oder später Objekten ist dies nicht mehr so einfach möglich. PHP bietet dazu spezielle Funktionen, die dies erleichtern.

Mittels `print_r($var [, $return = false])`¹⁵ wird der Inhalt einer Variable in menschenlesbarer Form ausgegeben. Vor allem im Zusammenhang mit Arrays ist dies sehr nützlich.

```
1 $var = ["test", "wert", "eintrag"];
2 print_r($var);
```

Dies erzeugt folgende Ausgabe:

```
Array ( [0] => test [1] => wert [2] => eintrag )
```

Soll der Rückgabewert nicht im Browser ausgegeben, sondern weiterverarbeitet werden, kann der optionale zweite Parameter auf `false` gesetzt werden.

⁸<https://www.php.net/manual/de/function.is-array.php>

⁹<https://www.php.net/manual/de/function.is-bool.php>

¹⁰<https://www.php.net/manual/de/function.is-float.php>

¹¹<https://www.php.net/manual/de/function.is-int.php>

¹²<https://www.php.net/manual/de/function.is-null.php>

¹³<https://www.php.net/manual/de/function.is-numeric.php>

¹⁴<https://www.php.net/manual/de/function.is-string.php>

¹⁵<https://www.php.net/manual/de/function.print-r.php>

Die Funktion `var_dump($var [, $var2, ...])`¹⁶ erzeugt eine detailliertere Ausgabe wie `print_r` und gibt auch den Typ des jeweiligen Eintrags mit an.

```
1 $var = ["test", "wert", "eintrag"];
2 var_dump($var);
```

Dies erzeugt folgende Ausgabe:

```
/var/www/html/wet2vo-examples/vo03/php-funktionen/variablen-funktionen.php:9:
array (size=3)
  0 => string 'test' (length=4)
  1 => string 'wert' (length=4)
  2 => string 'eintrag' (length=7)
```

3.4.2 Funktionen für Arrays

Für Arrays sind einerseits Funktionen zum Ermitteln der Größe oder zum Auffinden von Werten interessant, andererseits sind Funktionen im Zusammenhang mit Strings wichtig.

Größe eines Arrays

Die Funktion `count($array [, $mode])`¹⁷ (bzw. ihr Alias `sizeof()`) geben die Größe eines Arrays – also die Anzahl der darin enthaltenen Elemente – zurück. Wird der zweite, optionale Parameter `$mode` mit dem Wert `COUNT_RECURSIVE` angegeben, so zählt die Funktion rekursiv. Dies ist dann nützlich, wenn ein Array wieder andere Arrays enthält und die einzelnen Elemente aller Arrays summiert werden sollen.

```
1 $arr1 = [2, "Hallo", 4.3];
2 $arr2 = ["Test", 7, [3, 5, "Hi"]];
3 echo count($arr1); // 3
4 echo count($arr2); // 3
5 echo count($arr2, COUNT_RECURSIVE); // 6
```

Werte prüfen und zurückgeben

Die Funktion `in_array($needle, $haystack)`¹⁸ überprüft, ob sich ein gesuchter Wert (`$needle`) in einem Array (`$haystack`) befindet. Diese Funktion erspart ein manuelles Vergleichen in einer Schleife.

```
1 $arr3 = [1, 3, 5, 7];
2 in_array(3, $arr3); // true (1)
```

Mit `array_first($array)`¹⁹ und `array_last($array)`²⁰ gibt es seit PHP 8.5 die Möglichkeit, den ersten und letzten Wert im Array abzufragen.

```
1 echo array_first($arr3); // 1
2 echo array_last($arr3); // 7
```

¹⁶<https://www.php.net/manual/de/function.var-dump.php>

¹⁷<https://www.php.net/manual/de/function.count.php>

¹⁸<https://www.php.net/manual/de/function.in-array.php>

¹⁹<https://www.php.net/manual/en/function.array-first.php>

²⁰<https://www.php.net/manual/en/function.array-last.php>

Schlüssel prüfen und zurückgeben

Ebenso lässt sich abprüfen, ob ein Schlüsselwert existiert. Dazu dient die Funktion `array_key_exists($key, $array)`²¹. Ist der Schlüssel vorhanden, wird `true` zurückgegeben, ansonsten `false`.

```
1 $arr4 = ["name" => "John Doe", "alter" => 34, "geschlecht" => "männlich"];
2 echo array_key_exists("alter", $arr4); // true (1)
```

Mit `array_key_first($array)`²² und `array_key_last($array)`²³ können der erste und letzte Schlüssel im Array abgefragt werden.

```
1 echo array_key_first($arr4); // name
2 echo array_key_last($arr4); // geschlecht
```

Einen String in ein Array aufteilen

Mit der Funktion `explode($delimiter, $string)`²⁴ kann ein String zerteilt und als Array zurückgegeben werden. Der Teiler `$delimiter` ist eine beliebige Zeichenkette. Sie dient nur zur Trennung und scheint bei den Einzelteilen nicht mehr auf. Im folgenden Beispiel wird ein String mit einem Leerzeichen aufgeteilt.

```
1 $text = "Hello little world!";
2 $pieces = explode(" ", $text);
3 print_r($pieces); // Array ( [0] => Hello [1] => little [2] => world! )
```

Ein Array zu einem String zusammenfügen

Die Umkehrfunktion zu `explode()` stellt `implode($glue, $pieces)`²⁵ dar. Unter Angabe eines Verbindungselements `$glue` werden hier die einzelnen Einträge eines Arrays zu einem String zusammengefügt. Im folgenden Beispiel werden die Einträge eines Arrays in einen kommaseparierten String eingefügt.

```
1 $parts = ["name", "alter", "geschlecht"];
2 $string = implode(",", $parts);
3 echo $string; // name,alter,geschlecht
```

Einen String per Regex aufteilen

Anstatt wie bei `explode()` eine einfache Zeichenkette anzugeben, kann ein String auch anhand einer Regular Expression aufgeteilt werden. Dies ist mit Hilfe der Funktion `preg_split($pattern, $string)`²⁶ möglich. Details zu Regular Expressions folgen in Abschnitt 3.6. Das folgende Beispiel zerteilt ein Datum an typischen Trennzeichen wie Schrägstrich (/), Punkt (.) oder Bindestrich (-). Durch die Regular Expression kann angegeben werden, dass die Aufteilung erfolgt, egal welches der drei Zeichen im Datum vorkommt.

²¹<https://www.php.net/manual/de/function.array-key-exists.php>

²²<https://www.php.net/manual/de/function.array-key-first.php>

²³<https://www.php.net/manual/de/function.array-key-last.php>

²⁴<https://www.php.net/manual/de/function.explode.php>

²⁵<https://www.php.net/manual/de/function.implode.php>

²⁶<https://www.php.net/manual/de/function.preg-split.php>

```
1 $date = "13/03/2025";  
2 $values = preg_split("#[/.-]#", $date);  
3 print_r($values); // Array ( [0] => 13 [1] => 03 [2] => 2025 )
```

Arrays mit Callbacks durchsuchen

Mit PHP 8.4 sind weitere, mächtige Array-Funktionen hinzugekommen. Diese erlauben mit einer Callback-Funktion die Suchkriterien zu definieren:

- `array_find($array, $callback)`²⁷: Ein Element finden, das den Kriterien des Callbacks entspricht.
- `array_find_key($array, $callback)`²⁸: Einen Schlüssel finden, der den Kriterien des Callbacks entspricht.
- `array_any($array, $callback)`²⁹: Überprüfen, ob mindestens ein Element im Array den Kriterien des Callbacks entspricht.
- `array_all($array, $callback)`³⁰: Überprüfen, ob alle Elemente im Array den Kriterien des Callbacks entsprechen.

3.5 Formulardaten verarbeiten

Das Arbeiten mit *Formularen* gehört zu den wichtigsten Dingen im PHP-Workflow. Es stellt die einfachste Möglichkeit dar, Daten von Benutzer*innen abzufragen. Im Folgenden wird der serverseitige Ablauf besprochen, wie auf die eingegebenen Daten zugegriffen werden kann.

3.5.1 Vordefinierte Variablen

Für den Zugriff auf Formulardaten stellt PHP drei sogenannte *superglobale Arrays* zur Verfügung. Diese enthalten alle Daten aus einem abgeschickten Formular. Je nachdem, auf welche Art und Weise das Formular übertragen wurde (GET oder POST), liegen diese Werte in unterschiedlichen Arrays.

Übertragung mittels GET

GET ist eine HTTP-Übertragungsmethode (siehe Abschnitt 1.10). Um diese Methode zu verwenden, muss das Attribut `method` im Formularfeld den Wert `get` aufweisen:

```
1 <form method="get" action="get.php">  
2   <label for="infolabel">Info:</label>  
3   <input type="text" name="info" id="infolabel">  
4   <button type="submit">Abschicken</button>  
5 </form>
```

²⁷ <https://www.php.net/manual/de/function.array-find.php>

²⁸ <https://www.php.net/manual/de/function.array-find-key.php>

²⁹ <https://www.php.net/manual/de/function.array-any.php>

³⁰ <https://www.php.net/manual/de/function.array-all.php>

Wird dieses Formular abgeschickt, wird ein Request auf `get.php` erstellt und der Browser erhält vom Webserver diese als Response zurückgeliefert. Die Werte des Eingabefelds kommen mit einem Fragezeichen angehängt in `key=value`-Form beim Request mit. So könnte der URL des obigen Formulars etwa dann `http://localhost:8080/wet2vo-examples/vo03/formulardaten/get.php?info=Test` lauten, falls das Wort „Test“ im Formularfeld eingegeben wurde. Relevant für den Schlüssel ist der Name des Formularfelds, der durch das `name`-Attribut (nicht `id`!) gesetzt wird. Weitere Schlüssel-Wert-Paare werden mit dem kaufmännischen Und (`&`) angehängt.

Um nun mit PHP auf diesen Wert aus dem Request zugreifen zu können, stellt PHP das superglobale Array `$_GET` zur Verfügung. Die Arraystellen erhalten ihren Namen von den Formularfeldern, demnach kann über `$_GET["info"]` der Wert aus dem Formularfeld ausgelesen werden.

GET-Parameter können auch einfach „so“ an eine URL angehängt werden und scheinen ebenfalls im Array `$_GET` auf.

Übertragung mittels POST

POST ist eine weitere, in HTTP vorgesehene Übertragungsart. Die Daten werden hier nicht im URL sondern gesondert übertragen, weshalb auch größere Mengen möglich sind. Darüber hinaus bleibt der URL unverändert und die übertragenen Daten werden nicht sofort preisgegeben. Ein POST-Formular muss beim Attribut `method` den Wert `post` aufweisen:

```
1 <form method="post" action="post.php">
2   <label for="info">Info:</label>
3   <input type="text" name="info" id="info">
4   <button type="submit">Abschicken</button>
5 </form>
```

Wird das Formular hier abgeschickt, lautet der URL z.B. `http://localhost:8080/wet2vo-examples/vo03/formulardaten/post.php` – es sind keine Parameter sichtbar. Für den Zugriff in PHP existiert das superglobale Array `$_POST`. Die jeweilige Arraystelle wird wiederum über den Namen des Formularfelds bestimmt, in diesem Fall ist der Zugriff also über `$_POST["info"]` möglich.

Übertragungsunabhängiges Array

Das superglobale Array `$_REQUEST` enthält sowohl die Daten von `$_POST` als auch von `$_GET` und kann somit universell eingesetzt werden. Generell sollte jedoch explizit `$_POST` oder `$_GET` eingesetzt werden, um Fehler und Sicherheitslücken zu vermeiden. Würde etwa im Code mit einer Schleife über `$_REQUEST` iteriert und die Werte ausgegeben, könnte einfach durch das Anhängen an die URL-Zeile ein weiterer GET-Parameter „eingeschmuggelt“ werden, der dann auch angezeigt wird, obwohl alle Formulare auf der Seite nur mit POST arbeiten.

3.5.2 Übergabe von Werten

Die Übergabe von Werten von Formularen nach PHP wurde bereits soeben angesprochen. Je nach Übertragungsart kann ein superglobales Array ausgelesen werden. An der

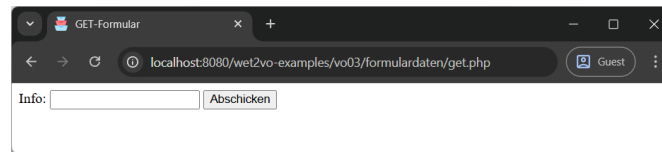


Abbildung 3.2: Die Seite zeigt nur das Formularfeld an. Da noch kein Inhalt eingegeben wurde, gibt der PHP-Teil nichts aus.

Stelle des Formularfeldnamens finden sich dann die Daten. Anbei zwei kleine Beispiele mit den beiden Übertragungsarten GET und POST.

```

1 <!DOCTYPE html>
2 <html lang="de">
3   <head>
4     <title>GET-Formular</title>
5     <meta charset="utf-8">
6   </head>
7   <body>
8     <form method="get" action="get.php">
9       <label for="infolabel">Info:</label>
10      <input type="text" name="info" id="infolabel">
11      <button type="submit">Abschicken</button>
12    </form>
13
14    <?php
15      if (isset($_GET["info"])) {
16        echo "<p>{$_GET["info"]}</p>";
17      }
18    ?>
19  </body>
20 </html>

```

Dieses Beispiel zeigt die Datei `get.php`. Sie enthält ein Formular mit einem Eingabefeld. Darunter ist ein kleiner PHP-Block, der überprüft, ob `$_GET["info"]` einen Eintrag enthält. Falls dies so ist, wird dieser ausgegeben. Zu Beginn wird nur das Formularfeld angezeigt, denn im superglobalen Array befinden sich noch keine Einträge (siehe Abbildung 3.2).

Danach wird der String „Hi“ eingetippt und das Formular abgeschickt. In der URL-Zeile wird der Parameter angehängt und `get.php` wird erneut geladen da das `action`-Attribut wieder auf die gleiche Seite verweist. Es hätte auch eine zweite, eigene PHP-Seite aufgerufen werden können.

Dieses Mal liefert `isset($_GET["info"])` den Wert `true` zurück und somit wird dieser Wert als Absatz ausgegeben. Abbildung 3.3 stellt dies dar.

Derselbe Ablauf lässt sich auch mit POST durchführen. Der Code dafür:

```

1 <!DOCTYPE html>
2 <html lang="de">
3   <head>
4     <title>POST-Formular</title>
5     <meta charset="utf-8">
6   </head>
7   <body>
8     <form method="post" action="post.php">

```

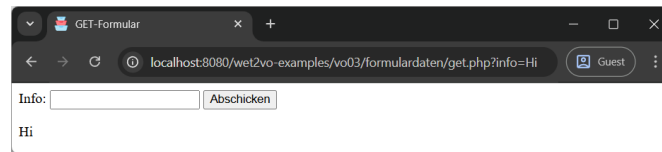


Abbildung 3.3: Nachdem das Formular abgeschickt wurde, wird der eingegebene Wert angezeigt.

```

 9      <label for="info" >Info:</label>
10      <input type="text" name="info" id="info">
11      <button type="submit">Abschicken</button>
12  </form>
13
14  <?php
15      if (isset($_POST["info"])) {
16          echo "<p>{"$_POST["info"]}</p>";
17      }
18      ?>
19  </body>
20 </html>

```

Geändert haben sich die Übertragungsart im Formular und der Zugriff über das superglobale Array `$_POST` (und die Datei heißt nun `post.php`). Die Ausgabe im Browser ist ebenfalls gleich, nur die angehängten Parameter in der URL-Zeile fehlen.

Kommen im Formularfeld mehrere Eingabefelder vor, funktioniert die Werteübergabe dafür analog. Wichtig ist, dass alle einzigartigen Namen im `name`-Attribut aufweisen, um sie unterscheiden zu können.

Anmerkung: Die ungeprüfte Übernahme und Weiterverarbeitung von User*innen-Input kann schwere Sicherheitslücken öffnen. Vorlesung 7 geht genauer auf diese Thematik ein.

3.5.3 Übergabe von Mehrfachauswahl

Im Falle von *Mehrfachauswahl* bei Checkboxes oder Multi-Select-Feldern muss eine leicht geänderte Strategie bei der Wertübergabe angewandt werden. In diesem Fall müssen alle Elemente denselben Namen aufweisen. Zusätzlich muss dieser mit abschließenden Array-Klammern (`[]`) versehen sein. Dies macht eine serverseitige Verarbeitung als Array möglich.

Das folgende Beispiel zeigt eine Mehrfachauswahl mit Checkboxes:

```

1  <!DOCTYPE html>
2  <html lang="de">
3  <head>
4      <title>Mehrfach-Auswahl</title>
5      <meta charset="utf-8">
6  </head>
7  <body>
8      <form method="post" action="checkboxes.php">
9          <p>Meine Hobbys:</p>
10         <label for="musik">Musik:</label>
11         <input type="checkbox" name="hobbies[]" value="musik" id="musik">

```

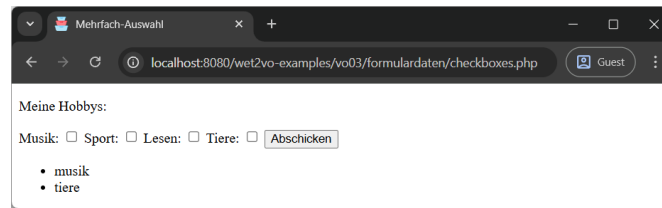


Abbildung 3.4: Die Checkboxes 1 (Musik) und 4 (Tiere) wurden ausgewählt. Die ausgegebenen Werte sind klein geschrieben, da sie im `value`-Attribut der jeweiligen Checkbox so angegeben sind.

```

12     <label for="sports">Sport:</label>
13     <input type="checkbox" name="hobbies[]" value="sport" id="sports">
14     <label for="reading">Lesen:</label>
15     <input type="checkbox" name="hobbies[]" value="lesen" id="reading">
16     <label for="animals">Tiere:</label>
17     <input type="checkbox" name="hobbies[]" value="tiere" id="animals">
18     <button type="submit">Abschicken</button>
19 </form>
20
21 <?php
22 if (isset($_POST["hobbies"])) {
23     echo "<ul>";
24     foreach ($_POST["hobbies"] as $hobby) {
25         echo "<li>$hobby</li>";
26     }
27     echo "</ul>";
28 }
29 ?>
30 </body>
31 </html>

```

Die Checkboxes haben alle den Namen `hobbies[]`, somit existiert in PHP ein Eintrag im superglobalen Array `$_POST["hobbies"]`, der ebenfalls wieder ein Array ist. Mit einer `foreach`-Schleife kann dann ganz einfach über das Array iteriert und jeder einzelne Eintrag in Form einer unsortierten Liste ausgegeben werden. Die Werte im Array werden von den Werten im Attribut `value` bei den Checkboxes bestimmt. Werden also Checkbox Nummer 1 (Musik mit dem Wert `musik`) und 4 (Tiere mit dem Wert `tiere`) ausgewählt, sieht die Ausgabe im Browser wie in Abbildung 3.4 aus.

3.6 Regular Expressions

Einige der Stringfunktionen in Abschnitt 2.3.9 von Vorlesung 2 boten eine simple Methode zu überprüfen, ob eine Zeichenkette in einer anderen vorkommt, oder nicht. Oft jedoch ist nicht bekannt, wie diese Zeichenkette genau aussieht. Es soll etwa nur überprüft werden, ob am Ende eines Strings zwei Zahlen stehen, es ist aber nicht klar, welche das sind, da sie von Eingaben der Benutzer*innen abhängen. In diesem Fall ist eine Überprüfung auf eine konkrete Zeichenkette nicht mehr möglich. Vielmehr muss auf ein Muster hin überprüft werden – in diesem Fall „zwei Zahlen am Ende des Strings“. Überprüfungen mit derlei Mustern werden *Regular Expressions* (kurz Regexp) oder zu

deutsch reguläre Ausdrücke genannt.

3.6.1 Regex-Funktionen

Reguläre Ausdrücke sind nicht allein auf PHP beschränkt. Es gibt sie in fast jeder Programmier- oder Skriptsprache. Noch dazu kommt, dass es verschiedene Syntaxen gibt, die verwendet werden können. In PHP herrscht die sogenannte *Perl-Syntax* vor.

Die erste Übereinstimmung finden

Um zu überprüfen, ob ein bestimmtes Muster in einem Text (d. h. einer Zeichenkette) vorkommt, wird die Funktion `preg_match($pattern, $subject [, &$matches])`³¹ benötigt. Hierbei ist `$pattern` das Muster, nach dem gesucht werden soll (siehe dazu Abschnitt 3.6.2), `$subject` ist die Zeichenkette, die durchsucht werden soll. Optional kann mit `$matches` noch ein dritter Parameter angegeben werden, der von der Funktion mit Teil-Suchergebnissen befüllt wird. Mehr dazu in Abschnitt 3.6.2 auf Seite 3-28. Die Funktion gibt 1 zurück, wenn eine Übereinstimmung gefunden wurde oder 0 falls nicht.

Alle Übereinstimmungen finden

Die Funktion `preg_match()` stoppt nach der ersten gefundenen Übereinstimmung. Sollen alle Vorkommen eines Musters erkannt werden, muss stattdessen auf die Funktion `preg_match_all($pattern, $subject [, &$matches, $flags])`³² zurückgegriffen werden. Die einzelnen Vorkommen können in `$matches` abgelegt werden (siehe Seite 3-28), `$flags` bestimmt dabei die Reihenfolge. Die Funktion gibt die Anzahl der Vorkommen als Zahl zurück.

Übereinstimmungen ersetzen

Soll nicht nur gesucht werden, ob ein Muster in einer Zeichenkette erkannt wird, sondern bei einer Übereinstimmung der gefundenen Text durch einen anderen ersetzen werden, kommt `preg_replace($pattern, $replacement, $subject)`³³ zum Einsatz. Die Parameter sind gleich, hinzu kommt lediglich `$replacement`, das Muster welches als Ersatz für die gefundene Zeichenkette dient.

Mit `preg_replace_callback($pattern, $callback, $subject)`³⁴ und dem ähnlichen `preg_replace_callback_array($patterns_callbacks, $subject)`³⁵ gibt es zwei Funktionen, die statt eines konkreten Ersetzungsmusters eine eigene Callback-Funktion aufrufen. Erste ruft dabei für ein Muster genau eine Funktion auf, letztere existiert seit PHP 7 und erlaubt es, ein Array mit Mustern und dazugehörigen Callbacks zu hinterlegen.

Beispiele mit Regex-Funktionen folgen im nächsten Abschnitt.

³¹ <https://www.php.net/manual/de/function.preg-match.php>

³² <https://www.php.net/manual/de/function.preg-match-all.php>

³³ <https://www.php.net/manual/de/function.preg-replace.php>

³⁴ <https://www.php.net/manual/de/function.preg-replace-callback.php>

³⁵ <https://www.php.net/manual/de/function.preg-replace-callback-array.php>

3.6.2 Muster definieren

Um ein zu suchendes *Muster* zu definieren, gibt es eine spezielle Syntax. Ein Muster ist immer eine Zeichenkette, steht also unter Anführungszeichen. Zudem verlangt die Perl-Syntax, dass am Anfang und Ende des Musters ein identisches Begrenzungszeichen steht. Dieses Zeichen kann frei gewählt werden, sollte aber im Muster nicht vorkommen. In der Regel werden zwei Schrägstriche (/) gewählt. Wird der Schrägstrich als Teil des Musters benötigt, kann z. B. auch die Raute (#) oder ein ähnliches Zeichen eingesetzt werden. Nur der Backslash ist nicht möglich. Er wird zum Escapen von reservierten Zeichen benötigt.

Eine Regular Expression sieht also prinzipiell wie folgt aus (statt den drei Punkten stehen dann die jeweiligen Muster-Zeichen, die im folgenden erklärt werden):

```
1 if (preg_match("/.../", $string)) { ... }
```

Einfache Zeichen

Soll mit einer Regular Expression nach einfachen Zeichen gesucht werden, genügt es, diese einfach im Muster zwischen den Begrenzungszeichen anzugeben:

```
1 "/a/"
2 "/test/"
```

Das erste Muster sucht, ob der Buchstabe „a“ vorkommt, das zweite sucht nach dem Muster „test“. Dies lässt sich jedoch mit String-Funktionen genauso effizient erledigen. Dennoch ein Beispiel:

```
1 preg_match("/a/", "Hallo Welt!"); // 1 (true)
```

Eine Sonderstellung nimmt der Punkt (.) ein. Er ist eine sogenannte Wildcard, steht also für jedes beliebige Zeichen. Ein Beispiel dazu:

```
1 "/.aus/"
```

Dieses Muster passt auf eine Reihe von Zeichenketten: „Maus“, „Haus“, „raus“ etc.

Soll explizit nach einem Punkt-Zeichen gesucht werden (z. B. um zu überprüfen, ob ein Datum im europäischen Format TT.MM.YYYY geschrieben ist), muss der Punkt mit einem Backslash escaped werden.

```
1 "/\..aus/"
```

Dieses Muster überprüft explizit auf die Zeichenkette „.aus“.

Zeichenklassen

Zeichenklassen lassen im Gegensatz zu den eben gezeigten Beispielen bereits Alternativen zu. Es wird nicht nur nach „a“ sondern z. B. nach „entweder a oder b oder c“ gesucht. Eine solche Klasse wird in eckigen Klammern angegeben und kann frei definiert werden. Der Bindestrich (-) wird verwendet, um Bereiche anzugeben. Einige Beispiele:

```
1 "[abc ]/"
2 "[a-z]/"
3 "[0-5]/"
4 "[a-zA-Z0-9_]/"
```

Der erste Ausdruck passt auf alle Zeichenketten, die entweder a, b, c oder das Leerzeichen enthalten. Der zweite sucht nach allen Kleinbuchstaben von a bis z, der dritte sucht Zahlen von 0 bis 5, der vierte sucht alle Klein- und Großbuchstaben, alle Zahlen und den Unterstrich.

```
1 preg_match("/[abc ]/", "Hallo Welt!"); // 1 (true)
```

Zusätzlich gibt es noch eine Reihe an benannten Zeichenklassen, die das Definieren von Bereichen oft erübrigen. Diese Klassennamen werden zwischen zwei eckige Klammern und Doppelpunkte gesetzt, z. B. `[:digit:]`.

Folgende solcher Klassen sind definiert:

- `[:alnum:]`: Alle Zahlen und Buchstaben.
- `[:alpha:]`: Alle Buchstaben.
- `[:ascii:]`: Alle 128 ASCII Zeichen.
- `[:blank:]`: Leerzeichen oder Tabulator.
- `[:cntrl:]`: Diverse Steuerzeichen.
- `[:digit:]`: Dezimalzahlen.
- `[:graph:]`: Alle druckbaren Zeichen, ohne Leerzeichen.
- `[:lower:]`: Alle Kleinbuchstaben.
- `[:print:]`: Alle druckbaren Zeichen, mit Leerzeichen.
- `[:punct:]`: Alle druckbaren Zeichen, ohne Buchstaben und Zahlen.
- `[:space:]`: Leerzeichen.
- `[:upper:]`: Alle Großbuchstaben.
- `[:word:]`: Buchstaben, Ziffern und der Unterstrich.
- `[:xdigit:]`: Hexadezimale Zahlen.

Allerdings können diese benannten Zeichenklassen nur innerhalb einer Zeichenklasse verwendet werden, weshalb nochmals zwei eckige Klammern benötigt werden:

```
1 preg_match("/[[:alnum:]]/", "Hallo 2026!"); // 1 (true)
```

Sollen in Zeichenklassen bestimmte Mengen an Zeichen explizit ausgeschlossen werden, kann dies mit dem Zirkumflex-Zeichen (^) geschehen. Mit dem folgenden Muster kann nach Zeichenketten gesucht werden, die keine Selbstlaute enthalten:

```
1 "/[^aeiou]/"
```

Im Beispiel wird ein Muster gesucht, das mit einem großen „H“ beginnt, dann darf *kein* Vokal folgen, danach kommen „llo“. Dies kommt so nicht vor, also schlägt der Vergleich fehl. Stünde statt „Hallo“ das Wort „Hyllo“ oder „Hbllo“ dort, gäbe es einen Treffer.

```
1 preg_match("/H[^aeiou]llo/", "Hallo Welt!"); // 0 (false)
```

Zeichen am Anfang und Ende eines Strings

Die Spezialzeichen Zirkumflex (^) und Dollar (\$) haben eine spezielle Bedeutung. Steht ein Zirkumflex am Beginn eines Muster-Ausdrucks, so bedeutet dies, dass eine übereinstimmende Zeichenkette mit diesem Muster beginnen muss. Umgekehrt bedeutet ein

Dollarzeichen am Ende, dass der String mit diesem Muster enden muss. Hier einige Beispiele:

```
1 "/^Die/"
2 "/^[0-9]/"
3 "/er$/"
4 "/^ABC$/"
```

Der erste Ausdruck findet Strings die mit „Die“ beginnen, der zweite findet Strings, die mit einer Zahl beginnen, der dritte Ausdruck findet Strings, die mit „er“ enden, der vierte findet Strings, die mit „ABC“ beginnen und enden, d. h. genau aus ABC (und keinen weiteren Zeichen) bestehen.

Wichtig ist, `"/^[0-9]/"` und `"/[^0-9]/"` nicht zu verwechseln. Das erste Muster findet Strings, die mit Zahlen von 0–9 beginnen, das zweite findet Strings, die keine Zahlen von 0–9 enthalten. Das Zirkumflex-Zeichen hat in diesem Fall eine doppelte Bedeutung, je nachdem, ob es außerhalb oder innerhalb der eckigen Klammern steht.

Angabe der Häufigkeit

Oft ist es nötig, bei einem Teil einer Regular Expression angeben zu können, wie häufig dieser Teil auftreten kann. Dafür stehen zunächst drei Zeichen zur Verfügung:

- `?`: Der Ausdruck kann 0 oder 1 Mal auftreten.
- `*`: Der Ausdruck kann 0 bis n Mal auftreten.
- `+`: Der Ausdruck kann 1 bis n Mal auftreten.

Darüber hinaus kann in geschwungenen Klammern auch eine explizite Häufigkeit angegeben werden:

- `{m}`: Der Ausdruck muss genau m Mal auftreten.
- `{m,}`: Der Ausdruck muss mindestens m Mal auftreten.
- `{,n}`: Der Ausdruck darf höchstens n Mal auftreten.
- `{m,n}`: Der Ausdruck darf zwischen m und n Mal auftreten.

Die folgenden Regular Expressions zeigen einige Beispiele:

```
1 preg_match("/ca+t/", "caaaaaaat"); // 1 (true)
2 preg_match("/ca+t/", "ct"); // 0 (false)
3 preg_match("/ca?t/", "caaaaaaat"); // 0 (false)
4 preg_match("/ca*t/", "ct"); // 1 (true)
```

Der Modifikator gilt hier immer nur für das Zeichen, hinter dem er steht. In diesen Beispielen also für das „a“. Wird mit Zeichenklassen (also Bereichen von Zeichen) gearbeitet, gilt er natürlich für den gesamten Bereich.

```
1 "/[0-9]{3}-[0-9]{3}-[0-9]{4}/"
```

Dies wäre etwa eine Regular Expression, um US-Telefonnummern im Format xxx-xxx-xxxx zu erkennen. Es werden 3 beliebige Zahlen gefolgt von einem Bindestrich, gefolgt von drei Zahlen, einem Bindestrich und schließlich 4 Zahlen erkannt. Die Nummer 123-456-7890 wäre etwas korrekt, 12-3456-7890 hingegen nicht.

Gruppierung/Subpatterns

Mit Hilfe von Klammerung können Einzelzeichen auch gruppiert werden. Somit kann ein Modifikator auch auf mehrere Zeichen angewandt werden:

```
1 "/a(bc)*/"
```

In diesem Fall wird eine Zeichenkette gesucht, die ein „a“ enthält, und optional die zwei Zeichen „bc“. Das eigentliche Kriterium ist hier nur „a“, denn ob „bc“ vorkommt oder nicht, ist in diesem Fall ja egal. Darum ist auch ein String wie „ab“ korrekt („a“ kommt vor, „ab“ nicht).

```
1 preg_match("/a(bc)*/", "aaaaa"); // 1 (true)
2 preg_match("/a(bc)*/", "abc"); // 1 (true)
3 preg_match("/a(bc)*/", "bc"); // 0 (false)
4 preg_match("/a(bc)*/", "ab"); // 1 (true)
```

Ein weiterer Nebeneffekt der Klammerung ist, dass geklammerte Teile von der Funktion `preg_match()` in ein Array zerlegt werden. Dieses wird als dritter Parameter angegeben und dann automatisch befüllt. Das Array an der Stelle 0 enthält den gesamten Ausdruck, der gefunden wurde. Die Stelle 1 enthält den Inhalt der ersten Klammer, usw. Hier ein Beispiel:

```
1 if (preg_match("/([0-9]{1,2})\.([0-9]{1,2})\.([0-9]{4})/", "23.04.2026", $matches)){
2     echo $matches[0]; // 23.04.2026
3     echo $matches[1]; // 23
4     echo $matches[2]; // 04
5     echo $matches[3]; // 2026
6 }
```

Diese Regular Expression sucht europäische Datumsformate. Zunächst stehen 1–2 Ziffern, dann ein Punkt, dann wieder 1–2 Ziffern, ein Punkt und schließlich 4 Ziffern. Durch die Angabe des Parameters `$matches` (kann beliebig heißen), wird ein Array erzeugt. Die Regex wurde geklammert (jeweils um die Ziffern), somit wird für jeden Klammerninhalt ein eigener Eintrag im Array erzeugt und die einzelnen Teile sind leicht zugreifbar.

Alternativen

Mit der sogenannten Pipe (`|`) lässt sich eine Alternative erzeugen. D. h. entweder der eine oder andere angegebene Teil des Musters muss vorkommen. Einige Beispiele:

```
1 preg_match("/cat|dog/", "The cat bit me"); // 1 (true)
2 preg_match("/cat|dog/", "The dog bit me"); // 1 (true)
3 preg_match("/cat|dog/", "The rabbit bit me"); // 0 (false)
```

Vorsicht bei Klammerung und Spezialzeichen wie `(^)` und `($)`:

```
1 "/^cat|dog$/" // Muss mit cat beginnen oder dog enden
2 "/^(cat|dog)$/" // Muss entweder mit cat oder dog beginnen und enden
```

Escape-Sequenzen

Der Backslash dient nicht nur als reines Escape-Zeichen (um z. B. mittels `\.` tatsächlich den Punkt zu suchen), sondern ist auch die Grundlage für einige sogenannte *Escape-Sequenzen*. Dies sind entweder nicht druckbare Zeichen, generische Zeichen-Typen oder

Tabelle 3.1: Die wichtigsten Regular Expression-Zeichen im Überblick.

Zeichen	Bedeutung
<code>string</code>	Sucht genau das Wort „string“
<code>.</code>	Ein beliebiges Zeichen
<code>^</code> , <code>\$</code>	Anfang und Ende des Strings
<code>()</code> , <code> </code>	Gruppierung, Alternative
<code>*</code> , <code>+</code> , <code>?</code>	Wiederholungen ($0-n$, $1-n$, $0-1$)
<code>{m}</code> , <code>{m,n}</code>	Explizite Wiederholungen
<code>[]</code> , <code>[^]</code>	Zeichenklasse und ausgeschlossene Zeichenklasse
<code>\</code>	Zum Escapen von Zeichen wie z. B. <code>.</code>

auch sogenannte Bedingungen, die an einem Teil des Strings auftreten können (z. B. ein Wort ist zu Ende). Die folgende Liste stellt einige wichtige Escape-Sequenzen dar, weitere finden sich unter <https://www.php.net/manual/de/regexp.reference.escape.php>.

- `\n`: Ein Zeilenumbruch (nicht druckbares Zeichen).
- `\t`: Tabulator (nicht druckbares Zeichen).
- `\d`: Jede beliebige Dezimalzahl (Zeichen-Typ).
- `\D`: Jedes Zeichen, das keine Dezimalzahl ist (Zeichen-Typ).
- `\s`: Jede Art von Whitespace-Zeichen (Zeichen-Typ).
- `\S`: Jedes Zeichen, das kein Whitespace ist (Zeichen-Typ).
- `\b`: Eine Wortgrenze (Bedingung).
- `\B`: Keine Wortgrenze (Bedingung).

3.6.3 Muster-Übersicht

Zur besseren Übersicht stellt Tabelle 3.1 nochmals alle wichtigen Zeichen im Zusammenhang mit Regular Expressions dar.

3.7 Weiterführende Literatur

Wie auch bereits im ersten Teil empfiehlt sich [5] als das primäre Nachschlagewerk auf Englisch. Funktionen, Arrays und Konstanten sowie die Verarbeitung von Formularfeldern sind hier sehr gut beschrieben. Wichtig ist, die aktuelle Auflage zu verwenden, da die Vorgänger die mittlerweile nicht mehr gebräuchliche POSIX Regular Expression-Syntax beinhalten.

PHP-Grundlagen werden ebenfalls von [6] gut erläutert, leider wurde auf Regular Expressions in diesem Buch vollkommen verzichtet. [7] hingegen widmet den regulären Ausdrücken ein ganzes Kapitel und bietet natürlich auch detaillierte Erklärungen zu den Grundlagen.

Wer sich intensiv den regulären Ausdrücken widmen möchte und ein umfangreiches Nachschlagewerk sucht, dem sei mit [1] *das* Werk zu dem Thema ans Herz gelegt. Für schnelles Nachschlagen und praktische Beispiele eignen sich passend dazu [2, 4].

Wer Lösungen zu fortgeschritteneren Problemen im Zusammenhang mit Arrays, Funktionen, Variablen oder auch Formularen sucht, der ist mit [3] gut bedient.

Quellenverzeichnis

- [1] Jeffrey E. F. Friedl. *Mastering Regular Expressions. Understand Your Data and Be More Productive*. 3. Aufl. Sebastopol, USA: O'Reilly, 1. Sep. 2006 (siehe S. 3-30).
- [2] Jan Goyvaerts und Steven Levithan. *Regular Expressions Cookbook. Detailed Solutions in Eight Programming Languages*. 2. Aufl. Sebastopol, USA: O'Reilly, 27. Aug. 2012 (siehe S. 3-30).
- [3] Eric A. Mann. *PHP Cookbook. Modern Code Solutions for Professional Developers*. Sebastopol, USA: O'Reilly, 20. Juni 2023 (siehe S. 3-30).
- [4] Tony Stubblebine. *Regular Expression. Pocket Reference*. 2. Aufl. Sebastopol, USA: O'Reilly, 1. Aug. 2007 (siehe S. 3-30).
- [5] Kevin Tatroe und Peter MacIntyre. *Programming PHP. Creating Dynamic Web Pages*. 4. Aufl. Sebastopol, USA: O'Reilly, 31. März 2020 (siehe S. 3-29).
- [6] Thomas Theis. *Einstieg in PHP 8 und MySQL*. 15. Aufl. Bonn, Deutschland: Rheinwerk Computing, 5. Apr. 2023 (siehe S. 3-29).
- [7] Christian Wenz und Tobias Hauser. *PHP 8 und MySQL. Das umfassende Handbuch*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 3. Mai 2024 (siehe S. 3-29).